

COMP 8005

Assignment 3

Design

Winston Nguyen
A01297231
Feb 19th, 2026

What it does

Goal

Use a controller/worker architecture to create a distributed password cracking system for UNIX systems.

Controller

Accept command-line input for the controller: shadow file (file to copy), username, and port number. Uses the inputs to identify hashing algorithms, salt, and password hash and registers with workers to crack the password. The controller will send chunks of work to connected workers that request work. Then receives results to report timing and results. The controller will also send heartbeat requests to verify the workers status.

Worker

Accept command-line input for the worker: controller_host (hostname/IP address), and port number. Then registers with a controller and requests a cracking job. The worker will then crack the password with a given number of threads. After cracking, send results to the controller. The worker will also respond to heartbeat requests.

Success condition (observable outcome)

The worker must successfully crack the password, and the program must measure and report controller-side parsing time, job dispatch latency, worker cracking time, result return latency, and total end-to-end runtime. The program must then exit successfully with no error messages.

Invariant (one checkable rule)

The program never displays a message unless the input arguments are valid.

How I will test it

Valid Shadow File and Username

The program is invoked from the command line with an existing and valid shadow file and username

Steps:

1. Run the program with valid arguments
2. Observe standard output.

3. Observe program termination.

Expected result:

- The controller registers a worker successfully retrieves the cracked password and timing results. Then the program exits successfully.

What it proves:

- Valid arguments transitions the system from argument handling to processing to connecting/connected, retrieving results, displaying results, and then to clean termination.

Invalid Shadow File or Username

The program is invoked from the command line with a non-existing shadow file or username

Steps:

1. Run the program with a non-existing shadow file or username
2. Observe standard error output.
3. Observe program termination.

Expected result:

- The controller fails to locate the shadow file and password information from the inputted shadow file and username

What it proves:

- The controller requires an existing shadow file and username

Missing Arguments

The controller program or worker program is invoked with a missing argument

Steps:

1. Run the program with a missing argument
2. Observe standard error output.
3. Observe program termination.

Expected result:

- An error or usage message is displayed; The program exits after cleanup.

What it proves:

- The system detects missing required inputs and prevents progression to the processing state.

Controller Invalid Arguments

The program is invoked with a non-numeric, or negative port.

Steps:

1. Run the program with an invalid port value.
2. Observe standard error output.
3. Observe program termination.

Expected result:

- An error or usage message is displayed, and the program exits after cleanup.

What it proves:

- Invalid numeric input triggers an error transition, blocking successful execution.

Worker Invalid Arguments

The program is invoked with a non-numeric, or negative port. Or the program is invoked with an invalid IP address

Steps:

1. Run the program an invalid port value or IP address.
2. Observe standard error output.
3. Observe program termination.

Expected result:

- An error or usage message is displayed, and the program exits after cleanup.

What it proves:

- Invalid numeric input triggers an error transition, blocking successful execution.

Worker Host Not Found

The program is invoked with valid arguments, but the IP address will not be a working IP address.

Steps:

1. Run the program with valid arguments without running a host program.
2. Wait until the timeout timer ends.

3. Observe program termination.

Expected result:

- An error is displayed, and the program exits after cleanup.

What it proves:

- The worker requires a listening host to connect to.

Help Option

The program is invoked with a help option.

Steps:

1. Run the program with the help flag.
2. Observe standard output.
3. Observe program termination.

Expected result:

- Usage information is displayed, and the program exits without attempting to process or display messages.

What it proves:

- Explicit user requests for help bypass normal execution and terminate cleanly.

FSM

State meanings

Controller

State	Meaning
START	The program has begun execution, but no work has been performed.
PARSE_ARGS	Raw input arguments are being read and identified.
HANDLE_ARGS	Parsed arguments are being validated and converted into usable configuration.

USAGE	An error or misuse has been detected, and feedback is shown to the user.
READ_SHADOW	The shadow file is opened and read into memory.
INIT_LISTENER	The controller initializes its communication endpoint (e.g., socket) to accept worker connections.
WAIT_FOR_WORKER	The controller is passively waiting for a worker to connect.
REGISTER_WORKER	A connecting worker is authenticated/validated and officially registered with the controller.
WAIT_FOR_WORKER_REQUEST	The controller waits for the registered worker to explicitly request a cracking job.
VALIDATE_REQUEST	The worker's job request is validated (e.g., checking readiness, authorization, or capacity).
PREPARE_JOB	The controller prepares the cracking task (splitting workload, packaging hash, salt, algorithm, and range).
SEND_CRACK_JOB	The prepared cracking job is transmitted to the worker.
WAIT_FOR_RESULT	The controller blocks while waiting for the worker's cracking results.
RECEIVE_RESULTS	The worker's result data (success/failure, password, timing) is received.
REPORT_RESULTS	Final outcome and timing statistics are displayed or logged.
SEND_HEARTBEAT	The controller sends a heartbeat message to verify the worker is still alive.
RECEIVE_HEARTBEAT	The controller receives and validates the worker's heartbeat acknowledgment.
CLEANUP	All sockets, threads, and resources are gracefully closed.
EXIT	The program has terminated successfully or due to failure.

Worker

State	Meaning
START	The program has begun execution, but no work has been performed.
PARSE_ARGS	Raw input arguments are being read and identified.
HANDLE_ARGS	Parsed arguments are being validated and converted into configuration values.

USAGE	An error or misuse has been detected, and feedback is shown.
INIT_SERVICE	Worker initializes its service environment (network setup, configuration, resources).
INIT_THREADS	Worker creates and prepares cracking threads but does not start cracking yet.
CONNECT_CONTROLLER	Worker establishes a network connection to the controller.
REGISTER_WITH_CONTROLLER	Worker registers itself with the controller and announces readiness.
REQUEST_JOB	Worker explicitly requests a cracking job from the controller.
WAIT_FOR_JOB	Worker waits for the controller to assign a cracking job.
RECEIVE_JOB	Worker receives the job payload (hash, salt, algorithm, and assigned keyspace range).
ASSIGN_JOBS	Worker splits the assigned keyspace across its internal threads.
INIT_CRACKING	Cracking context is initialized (hash function setup, salt configuration, candidate generation setup).
ATTEMPT_CRACK	Each generated candidate password is hashed and compared against the target hash.
CHECK_MATCH	The computed hash is compared to the target to determine if the password is correct.
WORK_EXHAUSTED	Assigned keyspace range has been fully searched without finding a match.
RECEIVE_HEARTBEAT	Worker receives a heartbeat request from the controller.
SEND_HEARTBEAT	Worker sends a heartbeat acknowledgment back to the controller.
UPDATE_PROGRESS	Worker updates and tracks cracking progress (attempt count, percentage, timing).
SEND_SUCCESS	Successfully cracked password is sent back to the controller.
SEND_FAILURE	Failure result (no match found in assigned range) is sent back to the controller.
CLEANUP	Worker closes sockets, terminates threads, and releases resources.
EXIT	The program has terminated.

Transition table

Controller

State	Call (function)	Fail State	Success State
START	parse_arguments		PARSE_ARGS
PARSE_ARGS	handle_arguments	USAGE	HANDLE_ARGS
PARSE_ARGS	usage		USAGE
HANDLE_ARGS	read_shadow	USAGE	READ_SHADOW
HANDLE_ARGS	usage		USAGE
READ_SHADOW	parse_shadow	USAGE	INIT_LISTENER
READ_SHADOW	usage		USAGE
INIT_LISTENER	listen	USAGE	WAIT_FOR_WORKER
INIT_LISTENER	usage		USAGE
WAIT_FOR_WORKER	worker_connect	USAGE	REGISTER_WORKER
WAIT_FOR_WORKER	error		USAGE
REGISTER_WORKER	registration_success	USAGE	WAIT_FOR_WORKER_REQUEST
REGISTER_WORKER	registration_failed		USAGE
WAIT_FOR_WORKER_REQUEST	job_request	USAGE	VALIDATE_REQUEST
VALIDATE_REQUEST	request_valid	USAGE	PREPARE_JOB
VALIDATE_REQUEST	request_invalid		WAIT_FOR_WORKER_REQUEST
PREPARE_JOB	job_ready	USAGE	SEND_CRACK_JOB
SEND_CRACK_JOB	job_sent	USAGE	WAIT_FOR_RESULT
SEND_CRACK_JOB	send_failed		USAGE

WAIT_FOR_RESULT	result_received	USAGE	RECEIVE_RESULTS
WAIT_FOR_RESULT	heartbeat_interval		SEND_HEARTBEAT
WAIT_FOR_RESULT	error		USAGE
SEND_HEARTBEAT	heartbeat_ack	USAGE	WAIT_FOR_RESULT
SEND_HEARTBEAT	heartbeat_failed		USAGE
RECEIVE_RESULTS	process_results	USAGE	REPORT_RESULTS
RECEIVE_RESULTS	invalid_result		USAGE
REPORT_RESULTS	complete		CLEANUP
REPORT_RESULTS	usage		USAGE
USAGE	cleanup		CLEANUP
CLEANUP			EXIT

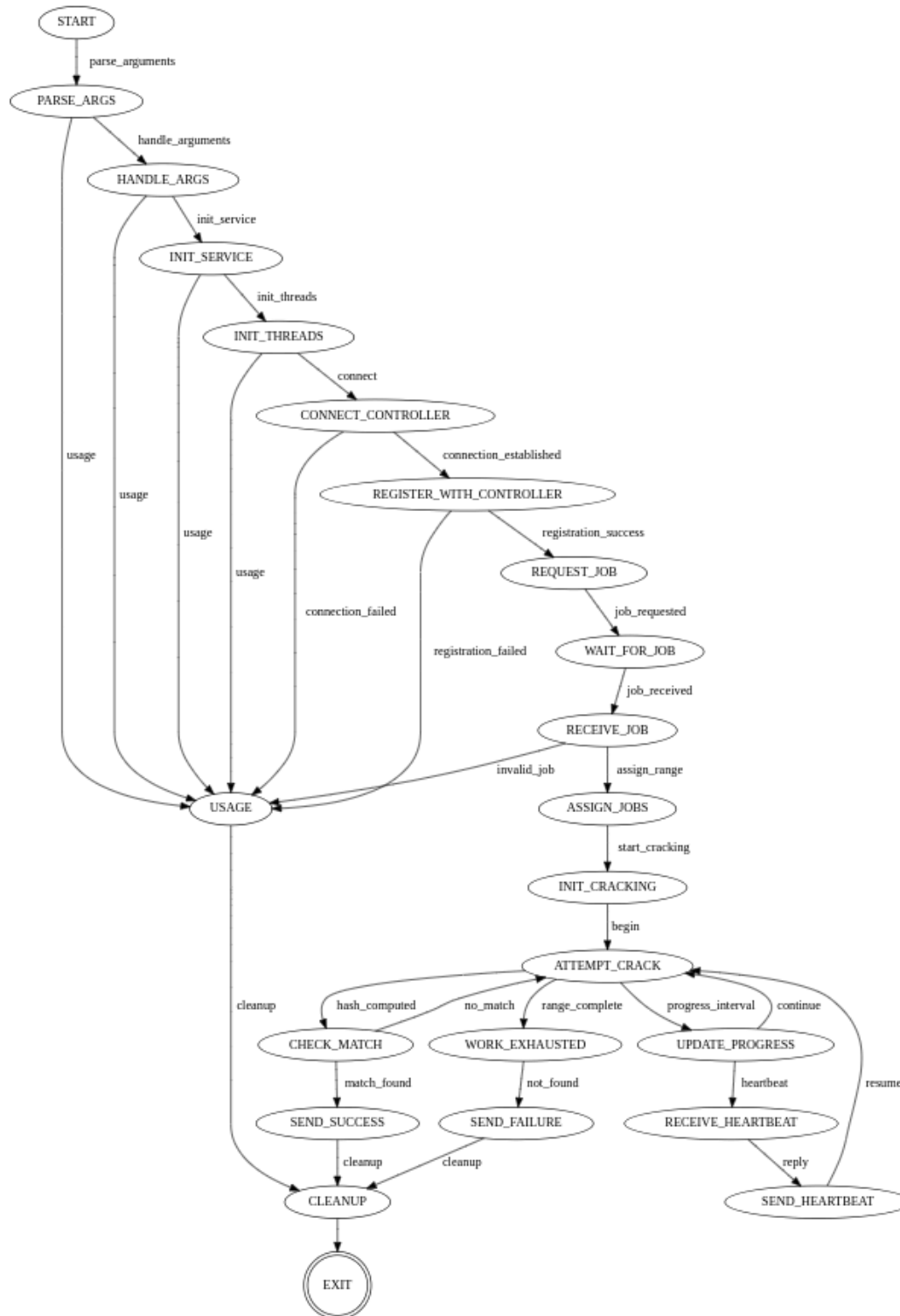
Worker

State	Call (function)	Fail State	Success State
START	parse_arguments		PARSE_ARGS
PARSE_ARGS	handle_arguments	USAGE	HANDLE_ARGS
PARSE_ARGS	usage		USAGE
HANDLE_ARGS	init_service	USAGE	INIT_SERVICE
HANDLE_ARGS	usage		USAGE
INIT_SERVICE	init_threads	USAGE	INIT_THREADS
INIT_THREADS	connect	USAGE	CONNECT_CONTROLLER
CONNECT_CONTROLLER	connection_established	USAGE	REGISTER_WITH_CONTROLLER
CONNECT_CONTROLLER	connection_failed		USAGE
REGISTER_WITH_CONTROLLER	registration_success	USAGE	REQUEST_JOB

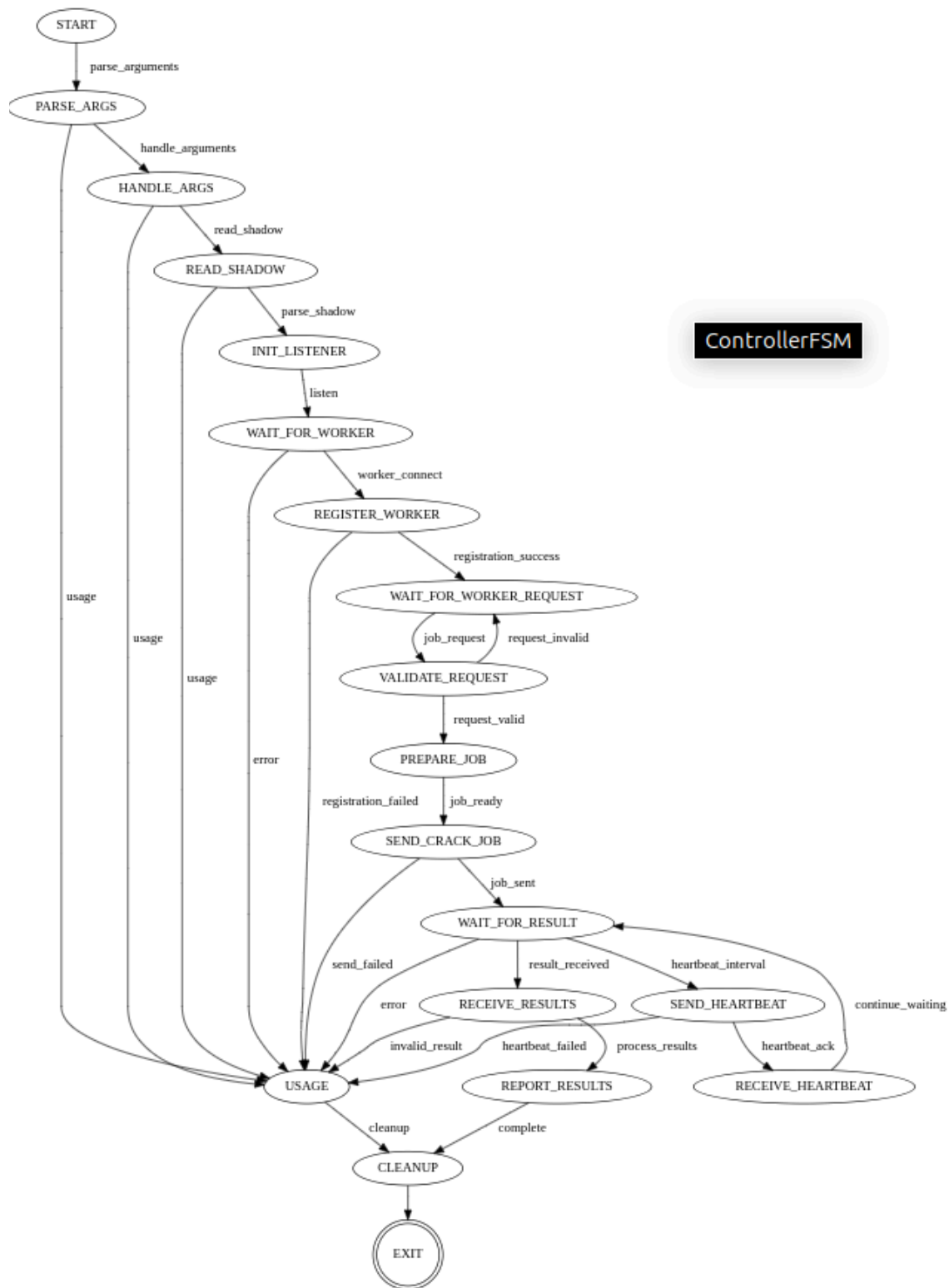
REGISTER_WITH_CONTROLLER	registration_failed		USAGE
REQUEST_JOB	job_requested	USAGE	WAIT_FOR_JOB
WAIT_FOR_JOB	job_received	USAGE	RECEIVE_JOB
WAIT_FOR_JOB	heartbeat		RECEIVE_HEARTBEAT
RECEIVE_JOB	assign_range	USAGE	ASSIGN_JOBS
RECEIVE_JOB	invalid_job		USAGE
ASSIGN_JOBS	start_cracking	USAGE	INIT_CRACKING
INIT_CRACKING	begin	USAGE	ATTEMPT_CRACK
ATTEMPT_CRACK	hash_computed	USAGE	CHECK_MATCH
ATTEMPT_CRACK	progress_interval		UPDATE_PROGRESS
ATTEMPT_CRACK	range_complete		WORK_EXHAUSTED
UPDATE_PROGRESS	continue		ATTEMPT_CRACK
UPDATE_PROGRESS	heartbeat		RECEIVE_HEARTBEAT
RECEIVE_HEARTBEAT	reply	USAGE	SEND_HEARTBEAT
SEND_HEARTBEAT	resume		ATTEMPT_CRACK
CHECK_MATCH	match_found		SEND_SUCCESS
CHECK_MATCH	no_match		ATTEMPT_CRACK
WORK_EXHAUSTED	not_found		SEND_FAILURE
SEND_SUCCESS	cleanup		CLEANUP
SEND_FAILURE	cleanup		CLEANUP
USAGE	cleanup		CLEANUP
CLEANUP			EXIT

FSM diagram

Controller



Worker



ControllerFSM

Functions

Controller

Name	Inputs	Outputs	Responsibility
parse_arguments	Raw command-line input	Parsed argument data	Identify options and message input
handle_arguments	Parsed argument data	Validated settings or error	Ensure inputs are valid and usable
usage	Error context	User-facing message	Inform the user of correct usage
read_shadow	Shadow file, username	Shadow file information	Opens and reads the shadow file
cleanup	Program context	Exit status	Ensure orderly termination
parse_shadow	Shadow file, username	Hashing algorithm, salt, password hash	Extracts the information from the shadow file
wait_worker		Connected worker	Connects a worker to the controller
dispatch_job	Hashing algorithm, salt, password hash		Sends cracking information to worker
wait_result		Worker results	Waits for a return from the worker
receive_result	Worker results	Cracking results	Receives the cracked password results
report_result	Cracking results	Results report	Gives a report based on the cracking results
send_heartbeat			Send a heartbeat request to the worker
receive_heartbeat		Worker progress	Receives the heartbeat response from the worker

Worker

Name	Inputs	Outputs	Responsibility
parse_arguments	Raw command-line input	Parsed argument data	Identify options and message input
handle_arguments	Parsed argument data	Validated settings or error	Ensure inputs are valid and usable
usage	Error context	User-facing message	Inform the user of correct usage
init_service		Connection service started	Starts up the socket to connect
cleanup	Program context	Exit status	Ensure orderly termination
register_controller	IP address, port	Established connection	Establishes a connection with the controller
wait_job		Cracking job	Waits for a job from the controller
start_cracking	Hashing algorithm, salt, hashed password		Initialize cracking content
crack_password	Hashing algorithm, salt, hashed password	Cracked results	Attempts to crack the password
check_match	Hashed password, cracked results	Cracked success or failure	Checks if the cracked results matches the password
success	Cracked results	Sends success results	Establishes successfully cracking

exhausted		Displays failure results	Work has been exhausted and cracking failed
failure		Sends failure results	
receive_heartbeat			Receives the heartbeat request from the controller
send_heartbeat	Worker progress		Sends the work progress to the controller